

UNIVERSITÄT POTSDAM

MASTER THESIS

TGT: A tool for automatic linguistic preprocessing

Author:

Alejandra Camelo Cruz

1st Supervisor:

Prof. Dr. Anna-Lena
Lamprecht

2st Supervisor:

Prof. Dr. Manfred Stede

*A thesis submitted in fulfillment of the requirements
for the degree of Master of Science in Computational Science*

December 17, 2025

Declaration of Authorship

I, Alejandra Camelo Cruz, declare that this thesis titled, “TGT: A tool for automatic linguistic preprocessing” and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed:

Date:

UNIVERSITÄT POTSDAM

Abstract

Faculty of Science

Department of Informatics and Computational Science

Master of Science in Computational Science

TGT: A tool for automatic linguistic preprocessing

by Alejandra Camelo Cruz

Linguistic research is slowed by time consuming tasks such as transcription, translation, and glossing, which create substantial bottlenecks in the research workflow. Within the LeibnizDream project, we introduce TGT, a research software tool that unifies NLP tools and provides an accessible local interface for automating linguistic preprocessing in environments with strict privacy and institutional constraints. Through a user friendly interface, TGT allows researchers to significantly accelerate linguistic research while adhering to a very specific workflow. The system abstracts away technical complexity, is fully usable by non technical users, and can be easily extended as additional languages or processing stages are required. Beyond the tool itself, this thesis contributes to Research Software Engineering in the humanities through a documented case study, usability driven insights, and an empirical evaluation of modern NLP models across multiple linguistic tasks. Overall, this work demonstrates that transparent and reflective research software is beneficial for research purposes and broadens the adoption of computational methods in the human sciences.

UNIVERSITÄT POTSDAM

Zusammenfassung

Naturwissenschaftliche Fakultät

Institut für Informatik

Master of Science in Computational Science

TGT: A tool for automatic linguistic preprocessing

by Alejandra Camelo Cruz

Linguistische Forschung wird durch zeitintensive Aufgaben wie Transkription, Übersetzung und Glossierung verlangsamt, die Engpässe im Prozess erzeugen. Im Rahmen des LeibnizDream Projekts stellen wir TGT vor, eine Software, die NLP Tools vereinheitlicht und eine zugängliche lokale Oberfläche für die Automatisierung linguistischer Vorverarbeitung in Umgebungen mit strengen Datenschutz- und institutionellen Vorgaben bereitstellt. Über eine benutzerorientierte Oberfläche ermöglicht TGT Forschenden, linguistische Arbeit zu beschleunigen, ohne von einem spezifischen Workflow abzuweichen. Das System abstrahiert technische Komplexität, ist vollständig von nicht-technischen Nutzenden verwendbar und lässt sich problemlos erweitern, sobald zusätzliche Sprachen oder Verarbeitungsschritte erforderlich werden. Über das Werkzeug hinaus leistet diese Arbeit einen Beitrag zur Research Software Engineering in den Geisteswissenschaften durch eine dokumentierte Fallstudie, nutzungsorientierte Erkenntnisse sowie eine empirische Bewertung moderner NLP Modelle über mehrere linguistische Aufgaben hinweg. Insgesamt zeigt diese Arbeit, dass transparente und reflektierte Software Forschungsvorhaben unterstützt und die Verbreitung computergestützter Methoden in den Humanwissenschaften erweitert.

Contents

Declaration of Authorship	i
1 Introduction	1
1.1 Motivation and problem setting	1
1.2 The role of software and NLP in linguistic research	2
1.3 TGT and the LeibnizDream Project	3
1.4 Structure of this work	4
2 Research proposal	5
2.1 Related work	5
2.2 Why TGT?	7
2.3 Research Question	7
3 Proposed approach	9
3.1 Development methodology	9
3.2 Stakeholders	10
3.3 Requirements	12
3.4 System Architecture	12
3.4.1 Technology Stack and development Timeline	12
3.4.2 Overview of System Components	16
Inference subsystem	16
Training subsystem	17
3.4.3 Summary of interaction between components	19
3.4.4 API Design and Architecture	19
3.4.5 Model selection, licensing and Open-Source	21
3.5 Testing	24
3.5.1 General Usability	26

3.5.2	Linguistic Specific Evaluation	27
4	Experiments	29
4.1	Glossing in LeibnizDream: linguistic background	29
4.2	Automatizing the glossing	31
4.3	Reconcilitating assumptions	32
4.4	Experimental setup	33
4.4.1	Data preparation	34
4.4.2	Training the models	35
4.5	Results	35
4.5.1	Yoruba	35
4.5.2	German	36
4.6	Analysis	37
5	Discussion	38
5.1	Planning	39
5.2	Development	40
5.3	Testing	41
5.4	Documentation	42
5.5	Distribution	42
5.6	Ethical considerations	43
6	Limitations and future work	44
7	Conclusion	46
8	Appendix	48
8.1	spaCy pretrained German	48
8.2	Stanza pretrained German	50
8.3	spaCy trained German (with vectors)	52
8.4	spaCy trained German (no vectors)	54
8.5	User stories	56
8.6	Requirements fullfilment	57
	Bibliography	58

List of Figures

2.1	High level workflow	8
3.1	Simple GUI prototype built with PySimpleGUI.	13
3.2	Simple GUI prototype built with javascript.	14
3.3	TypeScript prototype inference page	16
3.4	TypeScript prototype training page	17
3.5	TypeScript prototype main page	18
3.6	Component diagram of the system	19
3.7	Class diagrams for the inference component	20
3.8	Class diagrams for the training component	21
4.1	Glossing example from Comrie, Haspelmath, and Bickel, 2008	30

List of Tables

3.1	Users and stakeholders in LD project.	11
3.2	Requirements	12
3.3	Development approaches used in TGT with their respective pros and cons.	15
3.4	Open-source models used for linguistic tasks and their licenses	23
3.5	System Usability Scale (SUS) scores for all participants.	25
4.1	Example of UD feature to Leipzig Glossing Rule (LGR) map- ping for a simple German noun phrase.	32
4.2	Overview of evaluated models	33
4.3	Yoruba per-atomic-feature classification report.	36
4.4	German comparison of overall metrics across models.	37
8.1	Per-atomic-feature classification report for german spacy pre- trained model.	48
8.2	Per-atomic-feature classification report for german stanza pre- trained model.	50
8.3	Per-atomic-feature classification report for german trained model with vectors	52
8.4	Per-atomic-feature classification report for german trained model without vectors	54
8.5	Users stories gathered from stakeholders.	56
8.6	Mapping of system requirements to implementation details and fulfillment status.	57

List of Abbreviations

LGR	L eipzig G lossing R ules
LD	L eibniz D ream
NLP	N atural L anguage P rocessing
RSE	R esearch S oftware E ngineering
SE	S oftware E ngineering
SUS	S ystem U sability S cale
TGT	T ranslation G lossing T ransliteration
UD	U niversal D ependencies
UEQ	U ser E xperience Q uestionnaire

Chapter 1

Introduction

1.1 Motivation and problem setting

Of the approximately 7,000 languages spoken worldwide, only a small fraction are linguistically well studied. The predominant focus on so called WEIRD (Western, Educated, Industrialized, Rich, and Democratic) languages (Henrich, Heine, and Norenzayan, 2010) presents a significant issue, introducing a bias toward major world languages, especially English. This bias has far reaching implications for research in Linguistics, the Cognitive Sciences (Blasi et al., 2022) and in the field of Natural Language Processing (Joshi et al., 2020). More broadly, it reflects and legitimizes structural underrepresentation in science and technology (Ahia et al., 2023), restricting access to knowledge and technological advancement for non-WEIRD communities. Addressing this imbalance requires greater emphasis on studying languages other than English and main european languages.

However, language study is a complex and time intensive process that involves multiple stages requiring different levels of linguistic expertise. In the preprocessing stage, for example, tasks might include orthographic and phonetic transcription, where several hours may be needed to transcribe minutes of audio, morphosyntactic annotation, translation, etc. Furthermore, research efforts are often constrained by limited financial resources, inadequate infrastructure, and lack of trained personnel. This is especially challenging when the whole work must be accomplished by a very limited number of

people, many times only one researcher. Linguists working in such contexts face terrible bottlenecks in the collection, analysis, and archiving of linguistic data.

Overall, this heavy preprocessing workflow presents a major obstacle, as all of these steps must typically be completed before any specific data analysis can take place.

1.2 The role of software and NLP in linguistic research

Automating parts of this pipeline can substantially reduce the time and effort required for language study, and software can therefore play a significant role. While simple scripts can handle basic, repetitive tasks, Natural Language Processing (NLP) offers powerful tools for more complex steps. Technologies such as Automatic Speech Recognition (ASR), Named Entity Recognition (NER), part-of-speech (POS) and feature tagging, and machine translation can automate several stages of the process.

Nevertheless, existing tools are often fragmented and difficult to integrate. Their use must account for several factors, such as the meaningful combination of software components in unified systems, support for a diverse range of users, the decentralization and diverse quality of available tools, licensing constraints, and more. In Linguistics, such a software architecture can easily explode when trying to scale for multiple models across many languages.

On top of everything, there is a growing lack of trust and transparency surrounding computational tools, which include, amongst others, concerns about data privacy and data handling. This has critical implications when deploying technologies in research contexts, especially within societies, institutions and countries with strict, and increasingly stricter, regulations.

To make meaningful progress in language study under these constraints, it is essential to use existing tools and technologies thoughtfully. This requires

careful reflection on architectural, design and ethical choices to ensure that resulting systems are adaptable, scalable, accessible, and responsible.

1.3 TGT and the LeibnizDream Project

To address these challenges, particularly within the context of the LeibnizDream project (LeibnizDream Project, 2021), and, more broadly, to support language study across diverse resource settings, this thesis proposes TGT (Transcription, Glossing and Translation). TGT is an open-source, locally executable tool designed to assist with core linguistic preprocessing tasks such as transcription, transliteration, translation, and glossing. It provides a user friendly interface and is intended for use by both technical and non technical users.

The LeibnizDream project—funded by the European Research Council under the Synergy Grant 2019 call—is a collaborative effort between Humboldt University of Berlin (HU), the University of Milan-Bicocca (UniMiB), and the coordinating Leibniz-Centre General Linguistics (ZAS). Its goal is to develop a novel linguistic theory of language and its acquisition (LeibnizDream Project, 2021).

LeibnizDream conducts research on over 50 languages with a wide range of typological and resource profiles. These include high resource languages such as German, English, and Spanish; medium resource languages like Malayalam, Bengali, and Yoruba; and severely under documented languages such as Ecuadorian Siona (a Tucanoan language spoken in the Ecuadorian Amazon), K'iche' (a Mayan language spoken in Guatemala), and Karitiana (a Tupian language spoken in the Brazilian Amazon). Furthermore, most of the data collected in LeibnizDream consists of child language data, which makes both the data and its context highly sensitive.

TGT currently supports a subset of the languages studied within the LeibnizDream project and its primary utility lies in contexts inside LeibnizDream where personnel might be limited, and rapid, minimally supervised deployment is essential.

TGT's central goal is to provide an integrated environment for accessing a wide range of NLP tools relevant to linguistic data preprocessing. Rather than developing novel NLP tools from scratch, TGT emphasizes the smooth integration of existing pretrained models. This approach enables researchers and developers to use and combine complex computational workflows within a single and user friendly interface. In addition, TGT places strong emphasis on informed planning, architectural choices, and software decisions grounded in established principles of software engineering and research software engineering, bringing robust methodologies to the development of research software.

1.4 Structure of this work

This document aims to presents TGT, describing its architecture, potential applications, strengths and limitations, as well as opportunities for future improvement. Particular emphasis is placed on the lessons learned from and for the field of Research Software Engineering. The document is structured as follows:

- **Chapter 2** introduces related work, contextual requirements, and the research question.
- **Chapter 3** presents the development process, the final architecture of the system and qualitative testing
- **Chapter 4** presents the quantitative testing with experiments conducted with TGT for German and Yoruba.
- **Chapter 5** discusses Research Software Engineering in Linguistics and summarizes the findings.
- **Chapter 6** addresses limitations and proposes future directions.

The implementation of TGT is openly available at: <https://github.com/camelo-cruz/TGT>

Chapter 2

Research proposal

2.1 Related work

Several software tools for linguistic research already exist, offering essential functionalities through user friendly graphical interfaces. We review some of the most relevant examples, gradually moving toward approaches closer to our own and ultimately justifying the need for the approach proposed here. An extensive list of software for Linguistics until 2022 is listed in SIL (2022).

Praat (Boersma and Weenink, 2001) stands out as perhaps the most prominent tool for phonetic analysis. It is widely used by phoneticians and linguists to analyze and annotate audio data in a detailed phonetic manner. Praat provides a broad range of functionalities, including formant analysis, speech synthesis and manipulation, statistical analysis, and the creation of visual plots specifically made for phonetic research, etc. The software is platform independent and supports customization through both the Praat scripting language and integration with C or C++ code. Given its widespread adoption in the linguistic community, Praat has become an essential tool for phonetic research and is familiar to virtually every linguist.

Toolbox (SIL International, 2024a) and its successor, FieldWorks Language Explorer (FLEX) (SIL International, 2024b), both developed by the Summer Institute of Linguistics (SIL), are widely used in linguistic documentation and analysis. These tools support lexicon building and morphological analysis,

enabling linguists to construct dictionaries and annotate texts with interlinear glossing. FLEx additionally allows for the integration of cultural information and can automate glossing using stored grammatical data.

SayMore (SIL International, 2025), also developed by SIL, focuses on project and metadata management and is especially useful in fieldwork contexts. It simplifies the organization of recordings, participants, and sessions, and includes basic transcription and annotation tools.

In the multimodal research domain lies ELAN (The Language Archive, 2024), developed at the Max Planck Institute for Psycholinguistics. ELAN enables precise, time aligned annotations of audio and video recordings and is widely used in multimodal language research, including the documentation of sign languages and documentation tasks with video data. The software supports the creation of multiple tiers of annotation, allowing researchers to conduct analyses at various linguistic levels, such as phonetic, morphological, syntactic, or discourse, within a single aligned timeline.

Towards the integration of machine learning models in linguistic software, CoEDL Elpis uses modern models to facilitate the transcription of speech data, integrating both Kaldi and Hugging Face's wav2vec2 for automatic speech recognition (Foley et al., 2018). Like TGT, Elpis is specifically designed for non technical users and it offers an interface for training custom speech recognition models with minimal setup. The tool is implemented using Python and JavaScript and is deployed as a web based application, making it accessible and platform independent.

A system closely related to our approach is LiFE (Singh et al., 2023), an open source platform designed to bridge the technological gap in Linguistics and address the underrepresentation of languages. LiFE provides a comprehensive environment directed to the needs of field, theoretical, and computational linguists by supporting a range of processing tasks and languages. The platform integrates state of the art machine learning models for linguistic analysis and offers support for different file formats, multimodal inputs, and customizable workflows.

2.2 Why TGT?

Given the availability of many other software solutions, one should question the need for alternatives such as TGT, which currently provides more focused and limited functionality.

While LiFE and TGT share similar goals, TGT was developed to address the specific needs of the LeibnizDream project, with a strong emphasis on local processing, specific integration of the project tools and full control over deployment. TGT avoids reliance on third-party servers due to ethical concerns (e.g., data privacy, specifically working with children linguistic data) and bureaucratic constraints (e.g., institutional data governance and funding limitations).

TGT was designed with future extensibility and local usability in mind. It can be deployed easily on personal computers or internal lab servers without requiring complex infrastructure. This makes it especially valuable in environments where connectivity is limited or where researchers must work with sensitive data offline. Having complete control over the tool also facilitates further development and adaptation in other contexts.

Thus, the development of a domain adaptable and highly controlled tool like TGT is justified in contexts where simplicity, portability, privacy and tight integration with specific domain requirements are essential. Our specific workflow is illustrated in Figure 2.1.

2.3 Research Question

Building on the problem definition, the background and the justification for a specific software solution, this thesis investigates the following research question:

RQ: How can reusable, extensible, and privacy conscious research software be designed to support linguistic preprocessing, integrating existing NLP tools in institutional and ethically sensitive settings?

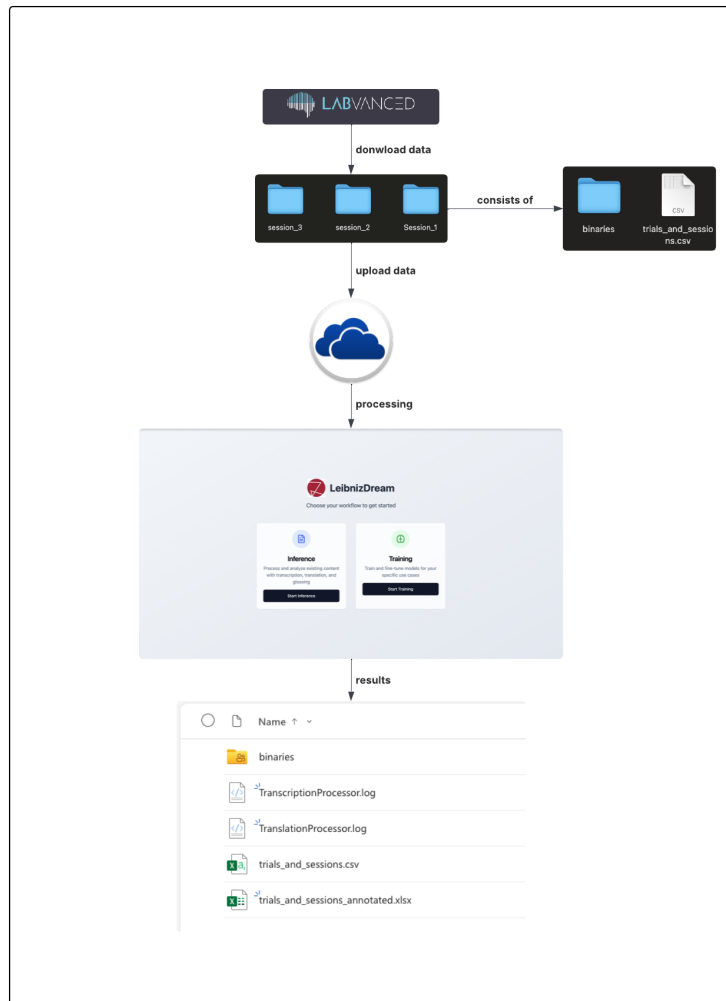


FIGURE 2.1: High level workflow

To investigate it, we document the development of TGT and gather insights from the empirical learnings earned throughout the design and implementation process. This approach follows established principles from Software Engineering and Research Software Engineering and reflect on our specific challenges. The findings presented here aim to provide an empirically grounded answer to the research question within the context of linguistic research software.

Chapter 3

Proposed approach

3.1 Development methodology

TGT was developed in a highly adaptive process to accommodate our context, which is characterized by evolving requirements and user needs. Traditional plan driven frameworks, exemplified by the waterfall model, propose a sequential flow of requirements elicitation, system design, implementation, and verification (Royce, 1987). These frameworks rely on a stable and well-defined set of requirements and are therefore not ideal to contexts like ours in which stakeholder and developer understanding evolves over time.

On the other hand, iterative and agile methodologies, along with DevOps practices, emphasize incremental delivery, continuous stakeholder feedback, responsiveness to change (Beck et al., 2001), and workflow automation. Such approaches generally align well with research contexts (Bajraktari, Binder, and Vogelsang, 2024), as they enable ongoing cycles of feature development, user validation, and code refactoring. Therefore, this project was more closely aligned with an agile methodology and development followed approximately weekly iterative cycles, during which new prototypes were continuously produced and evaluated.

Rapid documentation practices were integrated into each development phase. Informal requirements were gathered during regular team meetings and more structured inputs were collected at a later stage in the form of user stories.

An initial functional programming approach provided rapid implementations but became unsustainable as the number of supported languages and model dependencies increased. In response, the architecture was refactored towards modularity following Object Oriented Programming (OOP).

Recent advancements in AI-driven development tools were implemented to accelerate routine tasks: Builder.io, a low-code, AI-enhanced platform, facilitated rapid UI prototyping and generative language models (e.g., GPT-4, -5 and Claude) automated boilerplate code generation, refractored suggestions, and improved documentation (Alenezi and Akour, 2025). These technologies further reduced time for new feature development, relevant in a small development group such as ours.

In summary, the chosen development methodology combines principles from agile practices, DevOps workflows, rapid research oriented documentation, and AI assisted tooling. This approach is designed to maximize adaptability while keeping methodological strength. The following sections detail stakeholder identification, requirements elicitation, system architecture (Including API), licensing and testing.

3.2 Stakeholders

Stakeholders are individuals or organizations that can influence or be affected by a system, and they play a central role in defining its requirements (Pohl and Rupp, 2015). In our case, stakeholders included several categories of researchers and research assistants. Following stakeholders were identified, along with their type of contribution to the system:

1. **Principal Investigators (PIs):** Senior researchers who provide overall direction and coordinate the project. While they may not directly use the software, they set development priorities, allocate resources, and define expectations for the final output.
2. **Study Leads and Senior Researchers:** Based in the laboratories, these researchers specify how results should be processed and formatted for each research domain (study). They oversee entire experimental pipelines

across multiple languages and act as domain experts, though not necessarily experts in specific languages.

3. **Local Language Investigators:** Experts responsible for gathering and processing data in specific languages across countries and institutions. Their primary roles include collecting language data, reviewing and correcting it, providing feedback and evaluating the suitability of the software's output for their languages.
4. **Lab Managers:** Coordinate the data workflow, particularly for German (Berlin lab). They schedule data processing, assign tasks to student assistants, and enforce deadlines.
5. **Research Assistants:** The primary users of the software. They process raw linguistic data, annotate it, and make corrections based on domain, study, and language specific conventions. As main users, their feedback is crucial for improving usability and functionality.
6. **Data Manager:** A key non academic stakeholder who oversees the entire data pipeline. Responsible for ensuring data integrity, ethical compliance, and secure handling of sensitive information. The Data Manager uses the software to audit outputs and enforce data governance policies, acting as both a user and a gatekeeper of responsible and compliant data practices.

Table 3.1 presents an overview of the users, their level of interaction and their contribution to the requirements engineering.

Role	Requirements contributions	Interaction level
PIs	Lead the project; set goals	Indirect
Study Leads	Define domain constraints	Occasional
Local Investigators	Language expertise	Occasional
Lab Managers	Define deadlines; assign tasks	Frequent
Research Assistants	Primary users; evaluate quality, GUI	Frequent
Data Manager	Legal + ethical compliance	Frequent (audit)

TABLE 3.1: Users and stakeholders in LD project.

3.3 Requirements

After defining the stakeholders and understanding their roles, we elicited requirements. Requirements engineering is a fundamental phase of software development, as it defines what a system must achieve to fulfill its intended functions. It involves identifying the essential capabilities, constraints, and interactions that the software must support (Pohl and Rupp, 2015). In our project, requirements were continuously refined, typically during weekly meetings with stakeholders or whenever new needs emerged that could be addressed. Table 3.2 presents the full gathered requirements, and Appendix 8.5 provides an overview of their implementation.

ID	Requirement
F1	Accept input data in Labvanced format.
F2	Support additional input formats beyond Labvanced.
F3	Transcription, translation, transliteration, glossing across multiple languages.
F4	Customizable glossing detail (minimal vs full detail).
F5	Detect and report missing or incomplete data (e.g., insert [no audio]).
F6	Generate detailed logs of the process.
F7	Automatically generate required columns for processing.
F8	Annotate OneDrive session folders to indicate processing status.
NF1	Local data processing for privacy/security.
NF2	Minimal installation effort.
NF3	Low latency during use.
NF4	Cross platform support (macOS/Windows).
NF5	Always available without developer involvement.
NF6	GUI suitable for non technical users.
NF7	OneDrive integration to avoid manual handling.

TABLE 3.2: Requirements

3.4 System Architecture

3.4.1 Technology Stack and development Timeline

Building on the defined requirements, several prototypes were created and iteratively refined over time. We present the development timeline here and justify the architectural decisions that led to the current system design.

Phase 1 - CLI Prototype: The initial implementation was intended to be a simple Command Line Interface (CLI). Although this enabled rapid prototyping, it was highly impractical for non technical users due to setup and configuration challenges, including programming language versioning, installation of dependencies, general command line unfamiliarity, and limited discoverability, as it was accessible only through cloning the GitHub repository. Consequently, the CLI prototype was deprecated in favor of a more user friendly graphical interface.

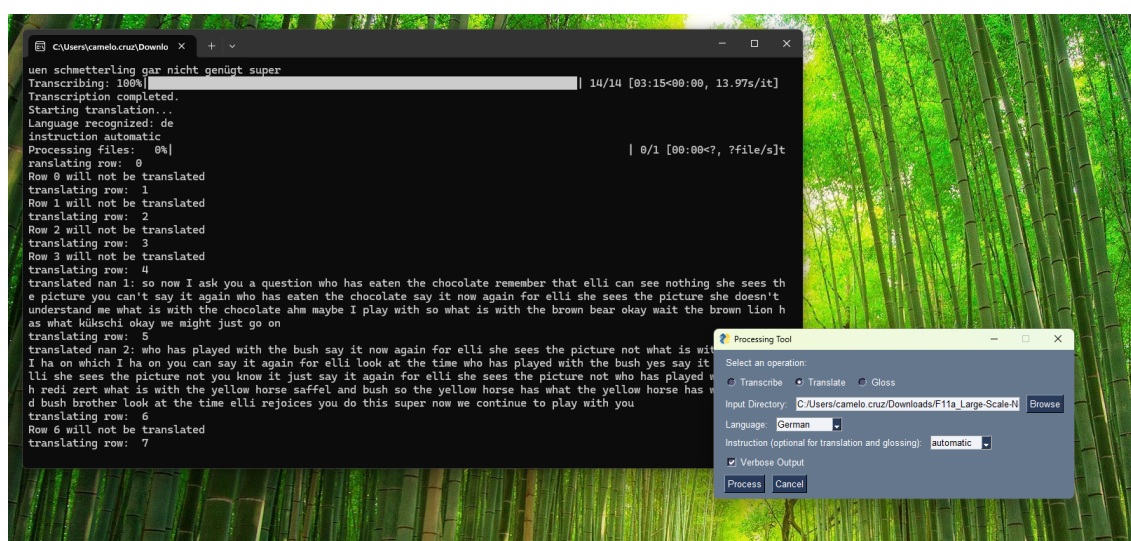


FIGURE 3.1: Simple GUI prototype built with PySimpleGUI.

Phase 2 - Standalone Desktop Application: To improve usability, a simple graphical interface was developed using first PySimpleGUI and later Tkinter. Build pipelines (via PyInstaller and GitHub Actions) produced executables for Windows and macOS, avoiding the need for users to install Python or manage virtual environments. However, dynamic model downloads required for large and many models were unreliable within the packaged binaries along with problems with hardware related variability.

Phase 3 - Containerized Deployment: A simple web frontend (JavaScript, HTML, CSS) was containerized alongside a Django backend, facilitating distribution via Hugging Face Spaces and Docker Hub. Adopting JavaScript

and Docker provided a standardized execution environment, solved problems related to model downloading inside a single executable and reduced OS compatibility issues, but introduced limitations: users had to learn how to use Docker and we struggled with sufficient compute resources and manual container management.

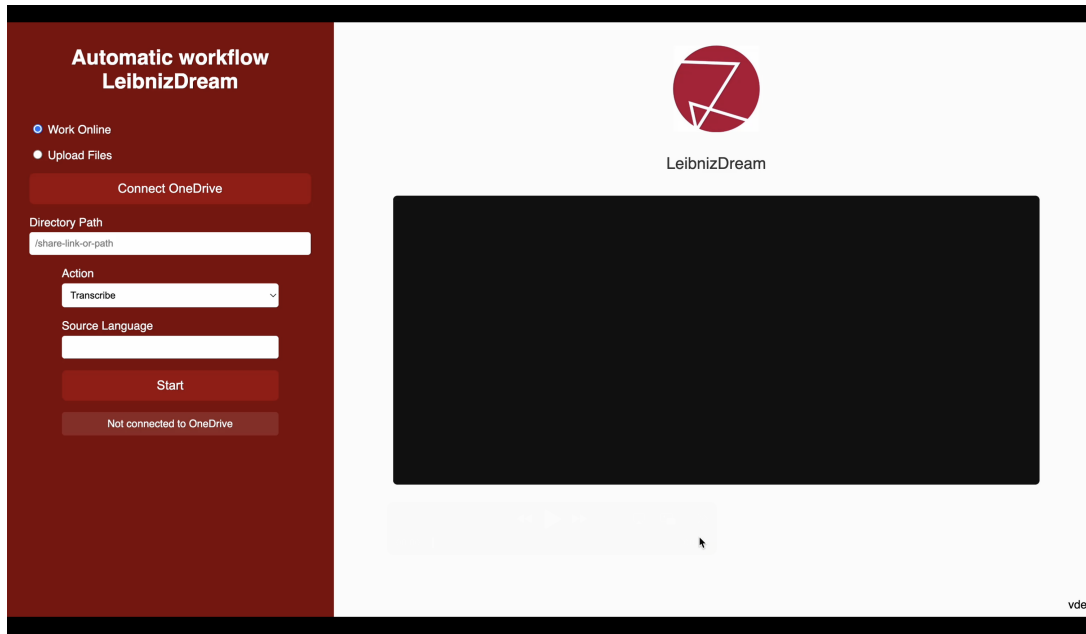


FIGURE 3.2: Simple GUI prototype built with javascript.

Phase 4 - Centralized Server with FastAPI: To improve maintenance, deployment was migrated to a local server accessible over the institute’s private network. This centralization enabled automatic updates, consolidated error monitoring, and basic access control. The backend framework was switched from Django to FastAPI to eliminate unnecessary overhead. The frontend was reimplemented in TypeScript using Builder.io.

Current Architecture Following the transition to a centralized server and FastAPI backend in Phase 4, the current system comprises a TypeScript frontend built with Builder.io and a Python/FastAPI backend, deployed on the

local server, with additional options to be used over Docker or as a CLI service. Previous versions remain archived in deprecated Git branches for reproducibility in the project's repository. Figures 3.5, 3.3 and 3.4 illustrate the current interface, and Table 3.3 compares the four development phases.

The current architecture of the system is based on OOP (Object Oriented Programming) principles. Key functionalities are encapsulated within reusable classes, which serve as modular components that can be easily extended, replaced, or modified without impacting the overall system. This modularity is critical to manage the scope of the project, which includes support for over 40 languages and integration of diverse NLP tools, ranging from multilingual models to highly language specific pipelines.

TABLE 3.3: Development approaches used in TGT with their respective pros and cons.

Comparison of Development Approaches in TGT		
Approach	Advantages	Limitations
CLI (Python)	- Simple to implement - No GUI - Lightweight - Aligns with functional programming	- Not user-friendly - Python versioning issues - Environment setup required - Requires CLI knowledge
PySimpleGUI / Tkinter + PyInstaller	- Quick GUI with few lines of code - Standalone builds - No need for user-side Python setup	- Large models difficult to download dynamically - Long builds - PyInstaller has hardware-specific issues
Django + JavaScript + Docker	- Structured backend - REST API support - System independent	- Heavy setup - Needs extra API implementation - Needs familiarization with docker - Excess boilerplate
FastAPI + JavaScript + Docker	- Lightweight - Async support - Less boilerplate - Easier frontend/backend coordination	- Manual UI work - Fewer built-in tools - More integration effort required - Extra setup for updates
Builder.io + FastAPI + Local server	- Simplified, clean and prettier frontend - Frontend easier to build + AI support - Server-deployed - Easy updates	- Builder.io dependency - More complex frontend/backend coordination

3.4.2 Overview of System Components

The system comprises two main subsystems: **Inference** and **Training**. The **Inference subsystem** uses pretrained models to perform the linguistic tasks, while the **Training subsystem** enables users to train their own models using custom data and predefined training protocols.

Inference subsystem

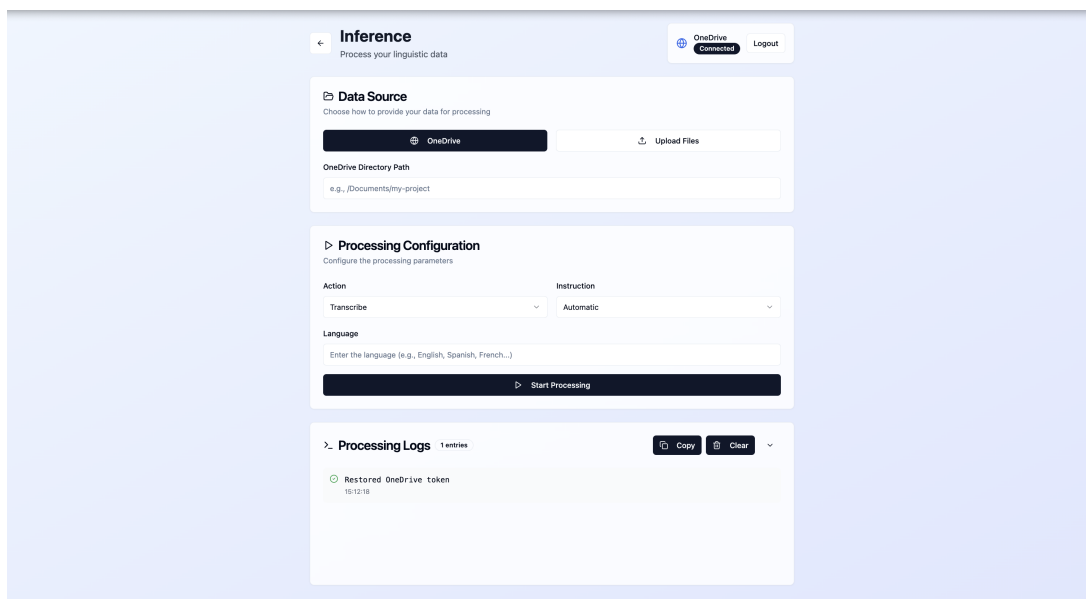


FIGURE 3.3: TypeScript prototype inference page

Linguistic tasks in the inference system follow a strategy pattern and represent the lowest layer of the architecture. Each task implements an abstract interface (e.g., `TranscriptionStrategy`, `TranslationStrategy`) that must be extended to introduce new functionality. These task classes define the specific data processing methods and apply them to either text or audio input.

Data processors implement a template method pattern, providing a consistent folder traversal mechanism while abstracting task specific operations such as reading files, transforming data, and generating output. A processor receives a folder as input, determines the appropriate strategy for the language, iterates through session subfolders, and processes the files. It executes

the selected linguistic task and returns the results for subsequent manual correction. Processors define which files to operate on and how they should be handled, ensuring a uniform workflow across different tasks.

At the top level, the **worker component** coordinates execution using a template-based design. It initializes the appropriate processor based on user specified inputs such as 1) target directory, 2) action (transcribe, translate, transliterate or gloss), 3) instruction (automatic, corrected or sentences) and 4) language.

A worker is the primary entry point for users, including API calls. In such cases, it receives a job object containing a job ID, a message queue, and a cancellation flag. The abstract `Worker` class defines a consistent interface for initiating concrete workers. This design enables clear definition for extension and usage in both the CLI and the API, across online and offline contexts.

Training subsystem

FIGURE 3.4: TypeScript prototype training page

The training subsystem is likewise initiated from a top-level worker class, mimicking the structure of the inference subsystem. The training worker receives three main inputs: (1) an input directory containing the data, (2)

a language identifier, which determines the appropriate training configuration, and (3) a study type, which will help users to further allocate the trained model.

The **preprocessor** defines how to gather and transform data so that it becomes suitable for the chosen training protocol. The preprocessed data is then passed to the **trainer**, which executes a specific training step and logs relevant metrics. In the training subsystem, the `Worker` class directly aggregates both the preprocessor and the trainer, in contrast to the inference subsystem where classes are organized hierarchically and build upon each other.

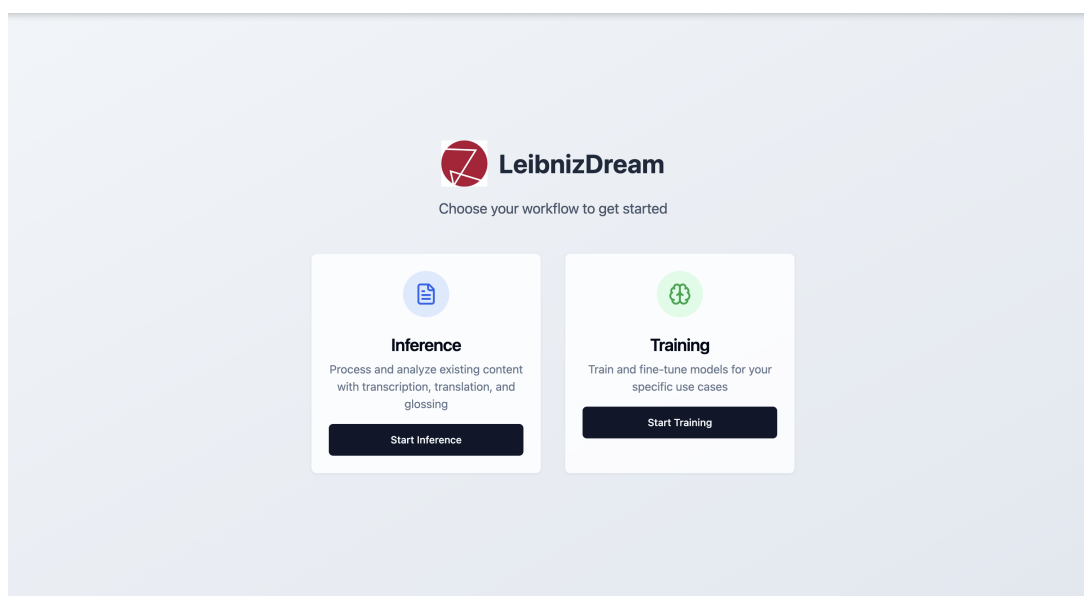


FIGURE 3.5: TypeScript prototype main page

This relationship reflects the loose coupling of components in the training pipeline: preprocessing and training can be executed independently, enabling reuse of preprocessed datasets and allowing the trainer to be applied to any other compatible dataset. The input of the trainer is a suitable dataset, it doesn't build it from inside. Each step in this process produces detailed logging and outputs for the user, including the generated dataset and the resulting trained model. Figure 3.6 illustrates the relationship between these components.

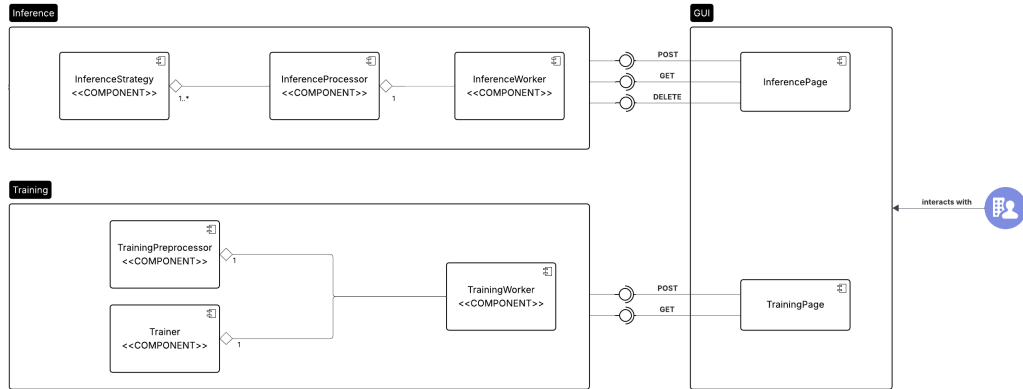


FIGURE 3.6: Component diagram of the system

3.4.3 Summary of interaction between components

In summary, the architecture is organized hierarchically and emphasizes a clear separation of responsibilities. **Workers** act as orchestrators, interpreting user inputs or API parameters. **Processors** and **preprocessor** manage directory traversal. Processors invoke the appropriate strategies to execute task specific logic and the preprocessor constructs the dataset required for model training.

In the inference subsystem, **strategies** encapsulate the core functionality for each linguistic task. Decoupling strategy logic from processing and orchestration enables the system to support new languages, replace models, or introduce new input formats with minimal changes. Finally, **factories** handle component instantiation, playing a central role in maintaining modularity by isolating and standardizing the creation process.

Figures 3.7 and 3.8 show specific details of the classes and how they relate to each other.

3.4.4 API Design and Architecture

The web application communicates with the backend through a small set of API endpoints. When a user starts a new processing task, the server creates a background job that runs independently of other jobs. Each job is centrally

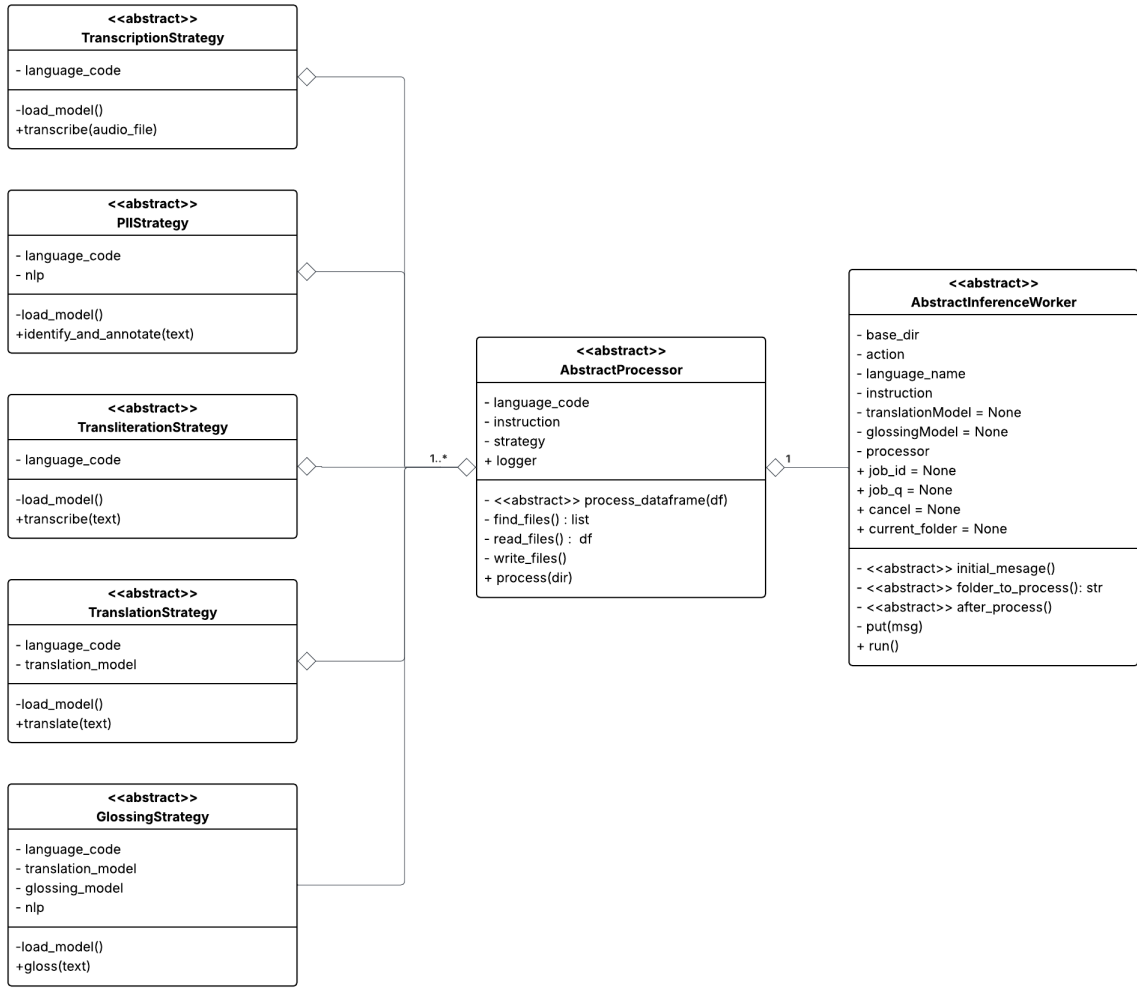


FIGURE 3.7: Class diagrams for the inference component

tracked, and a dedicated worker executes all required steps. Each subsystem is exposed through its own router, which serves as the entry point for frontend requests. Depending on whether the user is working with online or offline data, the API initializes either a OneDrive worker for cloud based processing or an offline worker for local processing.

The main entry point is the `POST /process` endpoint, which starts a new job using data from either a OneDrive share link or a local directory. The server returns a unique job identifier that the client can use to request progress updates. These updates are retrieved via the `GET /{job_id}/stream` endpoint,

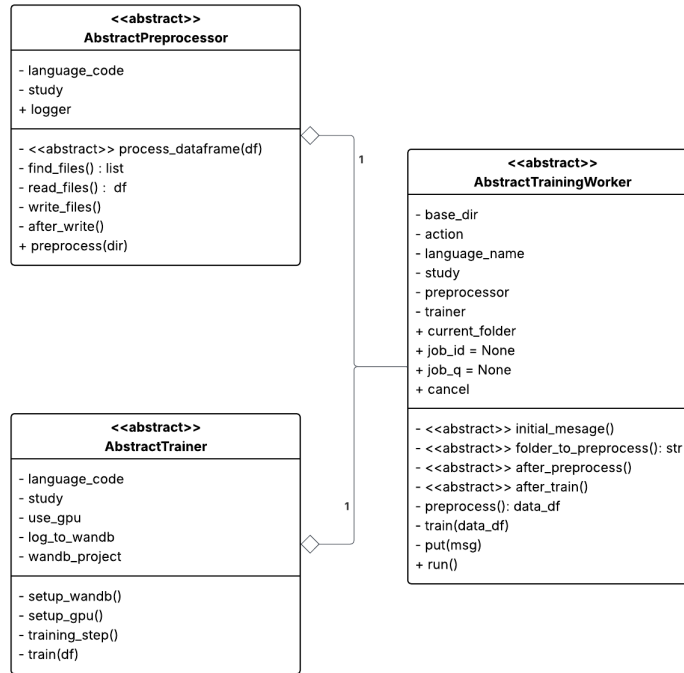


FIGURE 3.8: Class diagrams for the training component

which streams status messages back to the user interface until the job finishes or is cancelled. Once processing is complete, results are downloaded in the offline case as a ZIP file from `GET /{job_id}/download`. Users may also cancel a running job by sending a request to `POST /cancel`. Finally, the endpoint `GET /models/{task}` allows the client to query which models are available for a given task, such as translation or glossing.

3.4.5 Model selection, licensing and Open-Source

The system primarily relies on open-source libraries and pretrained models. The selection of these models was a critical design decision, as it directly influenced the system's output quality, latency, and resource demands. Preference was given to open-source and local models not only to avoid bureaucratic problems associated with proprietary software acquisition but also to retain control over data processing, particularly concerning the geographical location of servers.

This consideration was especially important in minimizing risks related to data handling and ensuring institutional and ethical guidelines. Moreover, the final goal was not to simply develop a functional system, but to allow its deployment across a variety of computing environments with constrained resources, such as personal computers, and institutional servers. Models were also selected not only for their accuracy but also for their efficiency and portability.

One outcome of this constraint was the deliberate decision to exclude large language models (LLMs) from the pipeline. Similarly, external APIs were avoided due to concerns over privacy, except in one case, the DeepL client. All selected models were required to run in memory and perform reliably on standard hardware. This strategy shows that lightweight, open models can be applied effectively across a range of linguistic tasks.

For **transcription**, we employed WhisperX (Bain et al., 2023), which supports speaker diarization, although classical Whisper (Radford et al., 2023) is also used for Estonian, whisper-large-v3-bn, for Bengali and PhoWhisper (Le, Nguyen, and Nguyen, 2024) for Vietnamese. **Translation** was primarily handled using the DeepL client. Its inclusion was strictly limited to preprocessed input from which Personal Identifier Information (PII) had already been removed. In addition, open-source models, OPUS-MT (Tiedemann and Thottingal, 2020), M2M-100 (Fan et al., 2021), and NLLB200 (Costa-jussà et al., 2022), were used for comparison purposes and offer the potential for fine-tuning on domain specific datasets.

For **glossing**, two tools were evaluated: SpaCy and Stanza. SpaCy offers CPU optimized models for a variety of languages (Honnibal et al., 2017) and was better suited for the current setup because of its training requirements. Stanza, on the other hand, provides a broader range of pretrained models and supports training too (Qi et al., 2020) but had some practical constraints in our context (CoNLL-U format for training and neural models requiring more computational power). Both libraries also offer Named Entity Recognition (NER), which was integrated for **personal identifiers identification**. Finally, **transliteration** required the use of task specific libraries, selected based on

language and script requirements.

Table 3.4 summarizes the models used for each task, along with the corresponding license for each, to ensure transparency in their usage and to clarify the legal and ethical boundaries.

TABLE 3.4: Open-source models used for linguistic tasks and their licenses

Task	Model	License
Transcription	whisper	MIT
	whisperX	BSD 2-Clause
	whisper-large-v3-bn	Apache 2.0
	PhoWhisper	BSD-3-Clause
Translation	OPUS-MT	MIT
	M2M-100	MIT
	NLLB-200	CC BY-NC 4.0
Transliteration	PyPinyin (Mandarin)	Apache 2.0
	pykakasi (Japanese)	GPLv3
Glossing	SpaCy	MIT
	Stanza	Apache 2.0

The integration of these components need attention to licensing constraints. Each software dependency and model may be governed by different license agreements, which may impose a series of obligations or restrictions. Therefore, all possible components, especially external models, need to be subject to license review process prior to inclusion. This review verifies that license terms are compatible with the project’s intended distribution and that no incompatible requirements arise from license combinations. Adherence to this protocol ensures both legal compliance and the longterm sustainability of the software.

TGT is released under the Apache License 2.0, chosen for its permissive terms and broad compatibility with the majority of our dependencies.

One of our optional translation models (NLLB200) is distributed under the Creative Commons Attribution NonCommercial 4.0 license, which prohibits

commercial exploitation. Although NLLB200 is not given by default and the current software has no commercial intentions, users who choose to install or enable this model must explicitly agree to its CC BY-NC 4.0 terms. The README and accompanying documentation therefore clearly state that the core software remains Apache 2.0–licensed, while any use of the NLLB200 model is governed separately by CC BY-NC 4.0 and is restricted to non commercial applications.

3.5 Testing

After defining the full system architecture, we needed a way to evaluate its performance. To assess TGT meaningfully, we adopted a mixed approach consisting of:

1. **Qualitative testing:** We assessed perceived usability using the System Usability Scale (SUS) by Brooke (1996) and asked for targeted personal feedback from student assistants and linguists.
2. **Quantitative glossing evaluation:** We did experiments and measured quantitatively the glossing output by comparing predicted morphological annotations against a manually glossed gold standard.

The qualitative evaluation is reported in this section, and the quantitative glossing evaluation follows in Chapter 4.

In general, classical formal approaches to testing, like unit tests, are rather challenging for cases like ours since formatting conventions and the specific elements of child language make an exact gold standard substantially different from the machine output. Therefore, we aimed for a more pragmatic approach, namely, mixing a qualitative approach with usability questionnaires and personal user’s feedback, and a quantitative one when gold standards were already available.

The System Usability Scale (SUS), a widely used 10-item questionnaire on a five point Likert scale that gives a reliable overall usability score. SUS captures users’ subjective evaluation of effectiveness, efficiency, and satisfaction,

outputs a result between 0 and 100 and allows to assess very different systems in terms of their usability. SUS was selected over the UEQ due to its shorter format, which simplifies implementation and minimizes participant workload. Although shorter versions of the UEQ exist (e.g., the UEQ-S), that family of questionnaires still targets hedonic qualities (Schrepp, Kollmorgen, and Thomaschewski, 2023), which are irrelevant for the current analysis. Besides, SUS is well validated and studied, and shows correlations with these other measures (Schrepp, Kollmorgen, and Thomaschewski, 2023), making it the more appropriate choice for this study.

We implemented the questionnaires using Labvanced (Finger et al., 2017) and sent it to seven users who had contact with the latest version of the system and were within the institute. We were mostly interested in their experience since the internal deployment allowed access to all functionalities. Five of them assessed the system. The results are shown in Table 3.5.

Participant	SUS Score
1	75.0
2	87.5
3	80.0
4	92.5
5	92.5
Mean	85.5
Standard Deviation	7.79

TABLE 3.5: System Usability Scale (SUS) scores for all participants.

In general, the results show that TGT performs well, with most SUS scores above 80. This indicates a high level of perceived usability and satisfaction. Although there is some variability between users, the overall consistency of the results suggests that the system offers a generally positive user experience with an excellent interpretation of the results following Bangor, Kortum, and Miller (2009).

Participants who had previously used earlier versions of the system gave higher ratings, while new or less frequent users reported slightly lower scores.

This suggests two important points: first, that the iterative development of TGT has led to visible improvements in usability over time; and second, that users who are unfamiliar with the system may initially face challenges that could be improved through clearer onboarding or further refinement.

Personal feedback from the users is reported in terms of general usability and its linguistic specific evaluation.

3.5.1 General Usability

Users considered the tool intuitive for general use, and the iterative improvements across versions were viewed positively. However, they also emphasized the continued need for a better onboarding process.

Users reported increasing complexity as the system evolved, particularly regarding installation and setup. This conflicts with the intended goal of creating a continuously improving, accessible system for all users. Docker was identified as a major source of difficulty: users struggled with installation, account creation, and limited resources. However, it was also reported that these issues disappeared once the system was centrally deployed on the institute's server, avoiding the need for any kind of setup. Interaction was reduced to basic GUI usage via VPN access, which was rated very positively since it additionally removes the need for physical presence to use the system.

Interestingly, although our introduction of Docker was intended to simplify distribution, it became the primary source of problems.

During the initial command-line phase, users (none of whom had technical experience) reported anxiety about "breaking their computers". This worry likely comes from unfamiliarity with the command line and therefore low trust in its use. Besides, logs were reported as mostly opaque in this stage. With repeated use, users reported better understanding of the system logs and increase trust in the command line use, and this phase was retrospectively seen as a valuable introduction to running software in a more technical environment.

Above all, the graphical user interface was regarded as the most substantial improvement. Usability was consistently rated high, although there is a clear need for a better onboarding.

Several areas were signaled for improvement. Increasing the visual complexity of the interface reduced the clarity of logging, which users identified as the most urgent issue. They requested more transparent logs, clearer progress indicators, including percentages, and notification mechanisms to signal task completion. Additional concerns included unclear labels within the interface and insufficiently transparent connection states. OneDrive integration was particularly unclear due to frequent required logins and token refreshes, which were neither intuitive nor clearly communicated.

Despite these challenges, users agreed that the system provided substantial support for their workflow and considered it a crucial and valuable tool for the overall process.

3.5.2 Linguistic Specific Evaluation

Researchers report high quality output for several languages, especially in transcription and translation and mentioned that it substantially reduced their manual workload since most corrections were minor. However, as expected, this level of performance was not uniform across all languages.

Linguists highlighted the automation of secondary tasks (column creation, file handling, and the automatic highlighting of fields requiring manual correction) as one of the most valuable features. These functions removed a large amount of routine overhead, helping them to focus on content and analysis rather than file management details.

Contrary to our initial assumption that automated glossing or training on project specific data would be the most valued features (as they are the most complex from a computational and development perspective), researchers perceived the secondary tasks and automatic transcription as the most beneficial. They reported that transcription is the most time intensive step, as it requires constant switching between audio and annotation, which makes

the process slow. Moreover, automatic transcription not only accelerates this workflow but, together with its automatic translation, enables early access to transcribed data in unfamiliar languages, reducing dependence on domain experts in the initial stages of analysis.

Nevertheless, automatic glossing remains also a desired future improvement and researchers emphasized the value of domain specific finetuning. Tokenization issues were salient in Japanese, for instance, likely because rule-based tokenizers struggles in languages without clear orthographic boundaries. similar problems should be expected for languages with comparable structural properties and this highlights the need of models better suited for linguistic purposes.

Overall, the system accelerated transcription and translation significantly but requires further refinement in glossing quality to fully meet linguistics specific needs.

Chapter 4

Experiments

Alongside the qualitative testing, we performed quantitative experiments on the glossing component for Yoruba and German to evaluate the system empirically. This chapter presents and discusses the results of these experiments.

To offer a deeper understanding of the gains and limitations, we begin with a linguistic background on glossing, continue with a detailed explanation of the glossing pipeline within TGT, and conclude with the reported evaluation metrics.

4.1 Glossing in LeibnizDream: linguistic background

Glossing, or interlinear glossing, is a linguistic annotation method that aligns morphemes of a source language utterance with a corresponding grammatical analysis. It is typically presented in four tiers: (1) the original text, (2) a gloss line that provides morphological analysis, (3) a morpheme by-morpheme translation, and (4) a free translation of the utterance. This technique facilitates cross linguistic comparison and supports linguistic analysis.

Interlinear glosses follow established conventions such as the Leipzig Glossing Rules to ensure consistency and clarity across languages. The Leipzig Glossing Rules were developed by the Department of Linguistics at the Max Planck Institute for Evolutionary Anthropology (Bernard Comrie, Martin Haspelmath) and the Department of Linguistics at the University of Leipzig (Balthasar

Bickel) (Comrie, Haspelmath, and Bickel, 2008). These rules establish a set of ten core guidelines for the morphological and syntactic annotation of linguistic data, covering aspects such as the use of abbreviations, alignment between gloss and object language, and formatting conventions.

Figure 4.1 presents an example from Russian, taken from Comrie, Haspelmath, and Bickel (2008).

(5) Russian

My	s	Marko	poexa-l-i	avtobus-om	v	Peredelkino.
1PL	COM	Marko	go-PST-PL	bus-INS	ALL	Peredelkino
we	with	Marko	go-PST-PL	bus-by	to	Peredelkino

'Marko and I went to Peredelkino by bus.'

FIGURE 4.1: Glossing example from Comrie, Haspelmath, and Bickel, 2008

LeibnizDream establishes the Leipzig Glossing Rules as a standard, while allowing for specific modifications. Data intended for analysis must be glossed according to these rules, with adjustments made based on the needs of study leads, domain requirements or local investigators. For instance, certain projects may require the treatment of underspecified grammatical categories or may implement simplified glossing conventions to facilitate the annotation process.

In the case of LeibnizDream, sentences are given in the source language, free translation and glossing are provided and, in some cases, a morphological segmentation of the language utterance might be given too. The following is an example of Yoruba taken from LD.

(1) àgùntàn tí ó ñ fò lórí ilé

sheep	REL	3SG-TNS	PROG	jump	head-LOC	house
-------	-----	---------	------	------	----------	-------

'The sheep that is jumping on the roof of the house.'

Glossing is amongst the most demanding tasks in terms of domain expertise. A linguist producing morphosyntactic annotations must not only have a

good understanding in morphosyntactic theory but also in the specific source language. Consequently, glossing is both intellectually intensive and time consuming, underscoring the potential value of developing automated methods.

4.2 Automatizing the glossing

For producing automatic glossing, TGT uses two libraries: SpaCy and Stanza, both operating within the *Universal Dependencies* (UD) framework (Marneffe et al., 2021). UD provides a uniform inventory of POS tags, morphological features and syntactic dependencies enabling language scientists to describe different languages consistently. To align UD analyses with LGR conventions, TGT introduces a controlled mapping layer that converts universal features into standardized leipzig labels:

- Number=Plur $\leftrightarrow$ PL
- Number=Sing $\leftrightarrow$ SG
- Tense=Past $\leftrightarrow$ PAST
- Gender=Masc $\leftrightarrow$ M

Table 4.1 illustrates this mapping process inside TGT using a real utterance in German. We extract the lemmas and translate them. We also extract morphological features of each token based on the Universal Dependencies (UD) format and then map them approximately into the Leipzig Glossing Rules format.

By parsing a sentence with any UD-compatible parser, extracting the feature bundles, and applying a simple rule based mapping, one can automatically generate interlinear morpheme glosses that roughly conform to Leipzig conventions, eliminating the need to write language specific glossing rules and allowing us to apply robust and widely used libraries.

TABLE 4.1: Example of UD feature to Leipzig Glossing Rule (LGR) mapping for a simple German noun phrase.

Surface	UD Features	LD Gloss
Der	der{Case=Nom, Definite=Def, Gender=Masc, Number=Sing, PronType=Art}	ART-DEF-M-SG-NOM
linke	linker{Case=Nom, Degree=Pos, Gender=Masc, Number=Sing}	left-M-SG-NOM-POS
Hund	hund{Case=Nom, Gender=Masc, Number=Sing}	dog-M-SG-NOM

Translation: 'the left dog'

Besides, this mapping not only facilitates the generation of glosses from UD annotated corpora, but also enables the partial reconstruction of UD annotations from existing glossed resources. As a result, annotated data produced for glossing purposes can be repurposed for tasks such as model finetuning or domain specific training. This functionality supports scalable, semi automated glossing workflows, like the one used in TGT, particularly valuable for low resource languages and settings where annotated data is scarce.

4.3 Reconcilating assumptions

However, the **Leipzig Glossing Rules (LGR)** and **Universal Dependencies (UD)** follow different theoretical assumptions that need to be reconciled. UD treats the *word* as the basic unit of analysis, whereas the Leipzig Glossing Rules are oriented towards *morphemes*, requiring morphological segmentation and alignment of annotation features at the morpheme level. UD does not support this level of granularity, as it processes words as a whole, regardless of their internal morphological structure.

Despite this limitation, UD offers a robust and language neutral framework for representing grammatical features, one that is already used in NLP tools.

We argue therefore that UD can meaningfully accelerate the glossing process and provide more systematic annotations. We consider this a worthwhile tradeoff that prioritizes speed, scalability, and consistency over exhaustive morphological details. In many contexts, the ability to quickly produce broad, comparable annotations may be more valuable than the exhaustive morphological detail. Nevertheless, this limitation can be critical for certain research questions, and potential solutions will be discussed in Chapter 6.

4.4 Experimental setup

To evaluate the quality of glossing produced by pretrained models and to assess the feasibility of training custom models within TGT, we conducted experiments on two languages: German and Yoruba. Everything was conducted using TGT’s interface, which abstracts away model usage, data preparation and training procedures.

For German, pretrained models were available for both SpaCy and Stanza, enabling a direct comparison against manually glossed gold data. As training baselines, we first trained a SpaCy model without static word vectors and tried to fine-tune the existing pretrained pipeline. We then trained a model using TGT’s default configuration, which includes static word vectors, and compared the results across all models.

For Yoruba, no pretrained pipeline or word vectors were available, so we trained a model from scratch using the existing annotations. These two scenarios reflect realistic use cases we aimed to evaluate. Table 4.2 shows a summary of the comparisons:

TABLE 4.2: Overview of evaluated models

Framework	Model Type	Language(s)
TGT (SpaCy)	Pretrained model	German
Stanza	Pretrained model	German
TGT (SpaCy)	Trained model with static vectors	German
SpaCy	Trained model without static vectors	German, Yoruba
SpaCy	Fine-tuned pipeline	German

To measure performance, we report per-feature precision, recall, and F1 scores, as well as micro- and macro-averaged metrics. Additionally, we report subset accuracy (exact match of the full feature set per token) and Hamming loss (fraction of individual feature labels incorrectly predicted, missing or wrongly assigned).

4.4.1 Data preparation

The training dataset consists of the manually corrected corpora annotated by local investigators (the gold standard). From the complete set of annotated sessions, we selected the minimal training configuration permitted by SpaCy: 100 annotated examples (110 counting the dev partitioning in the dataset). This setup was designed to simulate an extreme low resource scenario and to evaluate whether such a minimal configuration, requiring only limited glossing effort, would still yield a model capable of generalizing effectively to unseen data from the same study. This all reflects real world conditions.

Before providing data to TGT's training system, the dataset must be cleaned and formatted to ensure compatibility with the expected training format. After this, the training ready data are provided to TGT. From this point on, all subsequent steps occur within TGT and remain hidden from the end user.

The Preprocessor component loads the input files and removes non relevant content such as sentence numbering and non standard punctuation. Common punctuation marks, such as commas, periods, and parentheses, are retained. The processor then builds a DataFrame containing cleaned sentences, tokens, and morphological annotations. TGT subsequently maps Leipzig glossing features to Universal features (UFeats) and validates token and feature subset alignment.

After this, TGT stores sentence-gloss pairs in an internal DataFrame, which serve as both a record for manual inspection and the primary input for the training step.

4.4.2 Training the models

TGT loads the preprocessed DataFrame, containing sentences and Universal Features (UFeats). The appropriate spaCy pipeline for training is initialized and TGT attaches the morphological tags to each token.

The architecture used is based on the default SpaCy configuration for the Morphologizer, which combines a MultiHashEmbed embedding layer for feature representation and a MaxoutWindowEncoder for contextual encoding. The tagging head follows the spacy.Tagger.v2 specification.

TGT's Trainer partitions randomly (random seed 42) the dataset into 90% for training, and 10% for development, as we had to do the best of the minimal configuration with minimal data. This partition is important to monitor generalization, tune hyperparameters, use early stopping, and detect overfitting with internal validation during training.

After the training is done, TGT automatically saves the model and offers it for further use inside the inference subcomponent. The trained model is used to conduct the evaluation on the remaining sessions within the study.

4.5 Results

4.5.1 Yoruba

The Yoruba experiment shows consistently strong performance across labels and global metrics. All label types observed at evaluation time were also present during training, and the single annotator setup yields highly consistent gold annotations. Per label F1 scores are uniformly high (mostly above 0.95), and the global metrics, macro F1 = 0.960, micro F1 = 0.986, indicate both balanced and weighted accuracy. Subset accuracy of 0.987 means that in 98.7% of tokens the entire set of atomic features was predicted exactly, while a Hamming loss of 0.003 implies an average error rate of just 0.3% per token label decision. Table 4.3 presents the detailed results for the Yoruba data.

TABLE 4.3: Yoruba per-atomic-feature classification report.

Class / Metric	Precision	Recall	F1-score	Support
3	0.984	0.953	0.968	256
FOC	1.000	1.000	1.000	15
GEN	1.000	0.787	0.881	61
LOC	1.000	0.919	0.958	124
NO_GLOSS	0.987	0.999	0.993	1820
PERF	1.000	0.750	0.857	4
PROG	0.991	0.997	0.994	334
REL	0.995	0.995	0.995	403
SG	0.984	0.953	0.968	256
TNS	0.985	0.979	0.982	195
Overall Metrics				
Accuracy (subset)	0.987			
Precision (macro)	0.993			
Recall (macro)	0.933			
F1-score (macro)	0.960			
Precision (micro)	0.988			
Recall (micro)	0.984			
F1-score (micro)	0.986			
Hamming loss	0.003			

4.5.2 German

In German, the best results are obtained with SpaCy models trained with static word vectors, which achieve the highest subset accuracy (0.832) and micro F1 (0.906), while also minimizing Hamming loss (0.016). These findings show that from scratch model training incorporating pretrained embeddings is the best for our case .

By contrast, finetuning the pretrained SpaCy morphologizer was ineffective. It cannot not adapt to our task specific annotation scheme because the pre-trained model uses a much more comprehensive UD based label set. As a result, development accuracy plateaued near 20%, and the model’s predictions remained essentially identical to those of the original pretrained pipeline.

Table 4.4 summarizes the overall results across all models, comparing pre-trained pipelines and training with and without vectors. Detailed classification reports can be found in Appendices 8, 8.1, 8.2, and 8.3.

TABLE 4.4: German comparison of overall metrics across models.

Model	Accuracy (subset)	Macro F1	Micro F1	Hamming loss
Pretrained Stanza	0.583	0.478	0.797	0.035
(TGT) Pretrained spaCy	0.601	0.564	0.809	0.034
Trained spaCy (no vectors)	0.810	0.540	0.888	0.019
(TGT) Trained spaCy (with vectors)	0.832	0.562	0.906	0.016

4.6 Analysis

The Yoruba experiment demonstrates that when the label inventory is consistent, TGT trained models can achieve near perfect performance. This confirms that even with limited data, TGT enables the development of high quality models for less resourced languages under a low technical setup.

The German results provide a complementary view. They show that direct fine-tuning of pretrained pipelines is ineffective when the target annotation scheme is different from the model’s assumptions and that training models from scratch is the best option, apart from being the most optimal one.

Importantly, these results illustrate that high quality glossing can be achieved with as few as 100 labeled instances and modest computational resources, which is very positive for low resource scenarios, very common in Linguistics. This all validates quantitatively our design choices.

Despite these positive outcomes, limitations remain. Most notably, reliance on word level UD representations does not fully capture the morpheme level segmentation required by the Leipzig Glossing Rules. Future work should explore better morphological models capable of higher linguistic detail.

Chapter 5

Discussion

Finally, after presenting the system and its evaluation, we analyze TGT within a broader context, considering general Software Engineering and Research Software Engineering (RSE), and the case of Linguistics.

Software Engineering applies engineering principles to the design, development, and maintenance of software. The field emerged to address the growing complexity of critical systems and to generalize methods for building reliable business, embedded and system software (Felderer et al., 2025). In classical software engineering, projects typically begin with a relatively clear understanding of the product, stakeholders and requirements can be identified early in the process, financial and human resources are planned, and development is often carried out in larger and structured teams.

While software engineering began to formalize in the 1960s, research software engineering (RSE) is a much more recent field, gaining recognition only around 2012 (Katz and Hettrick, 2023). RSE focuses on developing software for research purposes (Felderer et al., 2025). It builds upon the principles of traditional software engineering but adapts them to the specific conditions of research contexts.

Unlike classical software engineering, stakeholder definitions and requirements in RSE are often unclear and evolve throughout the project. Teams are also typically small, often consisting of just one person and developers are frequently scientists writing code, or software engineers collaborating closely

with researchers (Bajraktari, Binder, and Vogelsang, 2024). This dynamic often leads to a gap between software engineering knowledge and/or deep domain knowledge. As a result, research software can be rather unstructured, as developers might be learning (or not) software development principles or domain knowledge on the fly.

Most work in RSE has focused on the natural sciences (Bajraktari, Binder, and Vogelsang, 2024). Existing publications on linguistic software, such as Boersma and Weenink (2001), Black and Simons (2006) or Wittenburg et al. (2006) touch on aspects of software engineering but don't address stakeholder analysis, requirements engineering, architecture design, testability and maintainability in a systematic way.

Additionally, the role and responsibilities of research software developers are not clearly defined in RSE contexts (Bajraktari, Binder, and Vogelsang, 2024), and Linguistics is no exception. Software emerge as a side effect of computational linguistics tasks and data management rather than as a planned engineering activity. Overall, although software plays an essential role in linguistic research, dedicated research on RSE for Linguistics remains limited.

Motivated by this gap, we report our findings and offer a discussion based on our empirical insights. The aim of this discussion is to critically reflect on how the phases of research software development outlined by Scheliga et al. (2019) played out in our project, evaluating what worked, what did not, and which lessons can be generalized to similar research software engineering contexts.

5.1 Planning

We adopted agile and DevOps practices mostly because they aligned well with the evolving and changing nature of our project, which is common in RSE. However, since most of the development was carried out by a single developer, it is unrealistic in the practice to adhere completely to a specific framework.

Adopting elements from a development framework provided nevertheless a systematic way to structure the work. It comes with practical tools such as user stories, persona creation, prototyping, automated workflows and standardized tools for system modeling that help organize and structure development.

Defining stakeholders and clarifying requirements helped in shaping the software's features and assessed their real world impact through targeted feedback. Researchers, annotators, data managers, and research assistants are groups with different technical skills and expectations regarding interfaces, formats, and annotation details. Our results showed that developer driven choices do not always align with user needs and therefore explicitly identifying stakeholder groups and prioritizing their requirements is a need.

Nevertheless, requirements engineering is often overlooked in the context of Research Software Engineering (RSE) (Bajraktari, Binder, and Vogelsang, 2024) and participation from stakeholders is sparse. Therefore, informal and easy accessible ways of gathering requirements, such as GitHub wishlists or user stories were useful. In our case, after building on initially gathered informal requirements and following early user interactions with the software, we also contacted individual users directly to increase involvement.

Planning also enforces documentation which is valuable in research contexts where reproducibility and methodological decisions matter. Research software is part of the research method itself, so design decisions must be documented to ensure transparency and replicability.

5.2 Development

Early use of best practices like inline documentation, consistent interfaces, and argument parsers facilitated later development as complexity grows. RSE systems often evolve from simple scripts into complex architectures, so these practices should have a positive impact for other cases, as they did for us.

Modularity, separation of concerns, and decoupling are especially useful in linguistic software, as it has to account for many languages, writing systems, annotation schemes, etc. Each processing task had to be replaceable, so that language specific features could be developed or modified independently without impacting other components. Developers in similar contexts would benefit from formal object oriented training. Proper architectural design is crucial because research software development is often carried out by only one person, and the systems could become unusable once that person leaves.

Our case also shows that rapid prototyping enables early availability, increases user testing and creates an ongoing feedback loop that improves usability. We observed, however, that technical barriers must be low for prototypes to be useful. In similar RSE contexts, prototypes should therefore be evaluated not only for functionality, but also for usability.

Graphical user interfaces (GUIs) were rated by users as the most important feature. Developing one, however, requires additional attention: logging and labels must be clear, and users need onboarding. Architectural complexity increases when a single developer manages both backend and interface, which makes implementation challenging. Still, the feedback was clear and proper logging and clear usage instructions are a must.

Even early prototypes built with simple libraries (like Tkinter) allow this, and although GUIs are particularly relevant in the humanities, we think they increase user engagement in any domain.

Relying only on a command line interface, while simpler to implement, was frustrating for users, lowered trust in the system, and led many to avoid the tool entirely. When applied in practice, this was too heavy and essentially unusable for the most part of the users, making a CLI only approach rather unsuitable in contexts like ours.

5.3 Testing

Evaluation is difficult in Linguistics: multiple annotations may be theoretically justified and “ground truth” often depends on linguistic theory. This

makes traditional unit testing challenging, and our case with child language data requires particular expensive expert annotation.

Despite this, systematic testing is essential. Traditional testing can be complemented with pragmatic evaluation, including usability questionnaires (Brooke, 1996). Where reference data exists, as in our retrospective glossing case (Chapter 4), quantitative metrics provide objective measures.

A mixed testing strategy, quantitative testing with user centered evaluation, is therefore the most realistic approach in a context like ours. It is a good tradeoff between the difficulty of getting gold standards and the empirical validation. Importantly, personal feedback from users is crucial as developers often imagine the workflow differently and this can lead to an unproductive development.

5.4 Documentation

The architecture and underlying models changed continuously, so rapid documentation strategies were useful. Documentation consisted of version control issues, commits, and README files, which were familiar to researchers. Complementary UML diagrams, such as those presented in this document in chapter 3, improve technical transparency and facilitate understanding of the system's structure. More interactive forms of documentation, such as short videos or visual user manuals, are an important tool to further support the onboarding as shown by user evaluation.

5.5 Distribution

Centralized distribution was the most effective deployment model. Feedback indicates that avoiding local setup is highly beneficial for users. TGT is deployed as a web based service on an institutional server, which takes advantage of existing infrastructure, ensures efficient use of institutional resources, and simplifies maintenance. This setup also allows real time updates, centralized logging, and remote access, which was highly appreciated.

However, institutional distribution can restrict external access due to security and infrastructure policies. For this reason, keeping a flexible distribution is positive as well. We provide command line access for users requiring full control and reproducibility, and make Docker images available.

Our evaluation showed that Docker, despite being attractive from a development perspective, introduced the highest number of usability problems. What appears efficient for developers may not translate to actual accessibility for researchers. Direct communication with users is essential and deployment decisions must be shaped by real use and not only by technical convenience.

5.6 Ethical considerations

Research in the human sciences can involve highly sensitive personal data, and ethical constraints become even stricter when children are involved. For this reason, avoiding external APIs and maintaining full control over data processing and storage is needed. We consider the use of locally hosted models and services in these contexts a must, and developers need to have particular attention when working with vulnerable populations or legally protected material. Licensing conditions must also be evaluated carefully, as institutional and legal frameworks can restrict how software and models are distributed, shared, or deployed over time.

Chapter 6

Limitations and future work

The main limitations of our system come from the specific formatting of the output and the variable quality of available models, which is particularly low for underresourced languages. Although our architecture provides a flexible and modular framework for integrating different components, identifying suitable models is costly. A centralized model capable of handling the multiple preprocessing tasks, such as a Large Language Model (LLM), could represent a promising direction.

Although the inclusion of LLMs under the current architecture is simple given the modular design of the system, they were not employed due to computational constraints, need of deployment on users machines with limited resources, and ethical concerns surrounding commercial models. Nevertheless, we believe that LLMs could improve the quality of linguistic data preprocessing tasks. They have demonstrated strong performance across multilingual settings (Ginn, Hulden, and Palmer, 2024), and recent work (Elsner and Liu, 2025; Ginn, Hulden, and Palmer, 2024) has shown their potential for automating interlinear glossing, even for underrepresented languages.

However, in the specific case of glossing the ability of LLMs to handle morphology remains largely unexplored (Weissweiler et al., 2023). Evidence suggests that the morphologically suboptimal design of tokenizers, such as BPE, limits the capacity of LLMs to represent complex morphological structures (Bostrom and Durrett, 2020; Hofmann, Pierrehumbert, and Schütze,

2021) and it is known that tokenizer quality is strongly biased toward high-resource languages (Ahia et al., 2023), which affects not only the quality of generated outputs but also the financial possibility of using leading commercial models for most languages. Besides, for interlinear glossing LLMs still underperform state of the art supervised systems (Ginn, Hulden, and Palmer, 2024).

Given these limitations, task and language specific models, such as those currently implemented in TGT, may yield more accurate, controllable, and computationally efficient results, but more systematic studies comparing baselines and LLM performance are needed to better understand the trade-offs in contexts like ours, which involve software engineering decisions and real world institutional applications.

Besides, existing work on using LLMs for interlinear glossing has focused primarily in low resourced languages and systematic evaluations are still required to determine how effectively LLMs can produce reliable and linguistically meaningful annotations in well resourced languages as well, or in specialized domains such as child language research.

Future work should therefore include a systematic investigation into the use of LLMs for interlinear glossing, comparing their outputs against established baselines and assessing their suitability for both well resourced and under-represented languages, as well as for specialized domains such as child language.

Chapter 7

Conclusion

This work addressed the following research question:

How can reusable, extensible, and privacy conscious research software be designed to support linguistic preprocessing, integrating existing NLP tools in institutional and ethically sensitive settings?

To answer it, we developed TGT, a locally deployable system for preprocessing of multilingual linguistic data, and used its development and evaluation as an empirical case study in Research Software Engineering for the humanities.

The development of TGT showed that grounding research software in software engineering principles is beneficial. Modularity, architectural consistency, documentation, open version control, and usability oriented design contribute to robust systems. In particular, GUIs are crucial for adoption among users, and mixed evaluation strategies made it possible to validate the system meaningfully. Additionally, full control of the system is essential when working in ethically sensitive environments.

Empirical results show that high quality performance on demanding linguistic tasks is achievable even with custom models trained on limited data, and can be made accessible to a broad range of users. In this way, complex computational workflows become usable in low technical environments, accelerating and improving the research process.

Limitations remain. Capturing the full level of detail required for linguistic analysis was difficult, and future work should investigate more effective strategies for this. In this sense, the potential of Large Language Models for multilingual processing, while promising, must be examined more carefully, especially for linguistically specific tasks.

Ultimately, this thesis contributes:

1. A functional research tool (TGT), refined through expert feedback, designed for reuse, and deployable in institutional environments.
2. A mixed evaluation methodology combining quantitative metrics with qualitative user feedback.
3. Practical guidelines for developing research software in the humanities, with a discussion on planning, development, testing, documentation, deployment and ethical considerations.
4. An empirical case study in Research Software Engineering, which contributes to the RSE gap in the human sciences and linguistics.

TGT demonstrates that structured software engineering, even in resource limited academic settings, allows to build sustainable, extensible, and genuinely usable tools. The thesis therefore not only delivers a functional system, but also a methodological perspective on how research software can be designed in Linguistics: grounded in engineering principles, evaluated with real users, and shaped by the practical constraints of research in the human sciences.

Chapter 8

Appendix

8.1 spaCy pretrained German

TABLE 8.1: Per-atomic-feature classification report for german spacy pretrained model.

Feature	Precision	Recall	F1-score	Support / Value
1	0.968	0.968	0.968	31
2	0.692	1.000	0.818	9
3	0.966	0.953	0.960	833
ACC	0.648	0.852	0.736	291
CIRC	0.000	0.000	0.000	57
COMP	0.000	0.000	0.000	93
DAT	0.424	0.903	0.577	31
DECL	0.000	0.000	0.000	93
DEF	0.874	0.960	0.915	174
DEM	0.989	0.737	0.845	118
DET	0.000	0.000	0.000	205
F	0.744	0.889	0.810	199
IMF	0.000	0.000	0.000	1
IMP	1.000	1.000	1.000	1

Continued on next page

Feature	Precision	Recall	F1-score	Support / Value
INDF	0.939	0.969	0.954	32
INF	0.952	0.808	0.874	172
M	0.755	0.954	0.843	459
M/N	0.000	0.000	0.000	4
N	0.503	0.947	0.657	95
NOM	0.760	0.989	0.859	442
NOM/ACC	0.000	0.000	0.000	14
NOM/ACC/DAT	0.000	0.000	0.000	1
NO_GLOSS	0.734	0.524	0.611	510
PART	0.831	0.914	0.871	81
PASS	0.000	0.000	0.000	1
PAST	0.714	1.000	0.833	5
PL	0.250	0.842	0.386	19
POSS	1.000	0.875	0.933	8
PRO	0.983	0.781	0.871	534
REL	0.167	1.000	0.286	1
SG	0.825	0.963	0.889	1207
Overall Metrics				
Accuracy (subset)				0.601
Precision (macro)				0.539
Recall (macro)				0.640
F1-score (macro)				0.564
Precision (micro)				0.809
Recall (micro)				0.808
F1-score (micro)				0.809
Hamming loss				0.034

Note. During evaluation, unknown classes were ignored by the scorer: [ART, FIN, FOREIGN, GEN, IND, INT, POS, PRS, SUB].

8.2 Stanza pretrained German

TABLE 8.2: Per-atomic-feature classification report for german stanza pretrained model.

Feature / Metric	Precision	Recall	F1-score	Support / Value
1	0.967	0.935	0.951	31
2	0.750	1.000	0.857	9
3	0.967	0.964	0.965	824
ACC	0.640	0.875	0.739	287
CIRC	0.000	0.000	0.000	57
COMP	0.000	0.000	0.000	93
DAT	0.352	0.731	0.475	26
DECL	0.000	0.000	0.000	93
DEF	0.923	0.971	0.946	173
DEM	0.000	0.000	0.000	118
DET	0.000	0.000	0.000	204
F	0.712	0.792	0.750	197
IMF	0.000	0.000	0.000	1
IMP	0.000	0.000	0.000	1
INDF	0.941	1.000	0.970	32
INF	0.949	0.882	0.914	169
M	0.759	0.936	0.838	451
M/N	0.000	0.000	0.000	4
N	0.506	0.935	0.656	92
NOM	0.777	0.984	0.868	439
NOM/ACC	0.000	0.000	0.000	14
NOM/ACC/DAT	0.000	0.000	0.000	1
NO_GLOSS	0.725	0.506	0.596	506
PART	0.951	0.951	0.951	81
PASS	0.000	0.000	0.000	1

Continued on next page

Feature / Metric	Precision	Recall	F1-score	Support / Value
PAST	0.161	1.000	0.278	5
PL	0.184	0.737	0.295	19
POSS	1.000	1.000	1.000	8
PRO	0.983	0.780	0.870	528
REL	0.000	0.000	0.000	1
SG	0.826	0.957	0.887	1193
Overall Metrics				
Accuracy (subset)				0.583
Precision (macro)				0.454
Recall (macro)				0.546
F1-score (macro)				0.478
Precision (micro)				0.806
Recall (micro)				0.788
F1-score (micro)				0.797
Hamming loss				0.035

Note. During evaluation, unknown classes were ignored by the scorer: [ART, CARD, DEM/REL, FIN, FORM, GEN, IND, INT, INT/REL, MASC/NEUT, NEG, POS, PRS, SING, SUB].

8.3 spaCy trained German (with vectors)

TABLE 8.3: Per-atomic-feature classification report for german trained model with vectors

Feature / Metric	Precision	Recall	F1-score	Support / Value
1	1.000	0.613	0.760	31
2	0.000	0.000	0.000	9
3	0.926	0.968	0.947	833
ACC	0.853	0.856	0.854	291
CIRC	0.902	0.807	0.852	57
COMP	1.000	0.989	0.995	93
DAT	0.909	0.323	0.476	31
DECL	1.000	0.989	0.995	93
DEF	0.735	0.971	0.837	174
DEM	0.962	0.424	0.588	118
DET	0.789	0.946	0.860	205
F	0.951	0.874	0.911	199
IMF	0.000	0.000	0.000	1
IMP	0.000	0.000	0.000	1
INDF	0.958	0.719	0.821	32
INF	0.882	0.866	0.874	172
M	0.928	0.950	0.939	459
M/N	0.000	0.000	0.000	4
N	0.885	0.568	0.692	95
NOM	0.953	0.873	0.911	442
NOM/ACC	0.071	0.071	0.071	14
NOM/ACC/DAT	0.000	0.000	0.000	1
NO_GLOSS	0.859	0.969	0.911	510
PART	0.971	0.840	0.901	81
PASS	0.000	0.000	0.000	1

Continued on next page

Feature / Metric	Precision	Recall	F1-score	Support / Value
PAST	0.000	0.000	0.000	5
PL	1.000	0.211	0.348	19
POSS	0.000	0.000	0.000	8
PRO	0.994	0.860	0.922	534
REL	0.000	0.000	0.000	1
SG	0.969	0.948	0.958	1207
Overall Metrics				
Accuracy (subset)				0.832
Precision (macro)				0.629
Recall (macro)				0.537
F1-score (macro)				0.562
Precision (micro)				0.918
Recall (micro)				0.895
F1-score (micro)				0.906
Hamming loss				0.016

Note. During evaluation, unknown classes were ignored by the scorer: [ART].

8.4 spaCy trained German (no vectors)

TABLE 8.4: Per-atomic-feature classification report for german trained model without vectors

Feature / Metric	Precision	Recall	F1-score	Support / Value
1	1.000	0.323	0.488	31
2	0.000	0.000	0.000	9
3	0.901	0.956	0.928	833
ACC	0.866	0.797	0.830	291
CIRC	0.818	0.789	0.804	57
COMP	1.000	1.000	1.000	93
DAT	1.000	0.258	0.410	31
DECL	1.000	1.000	1.000	93
DEF	0.782	0.948	0.857	174
DEM	0.925	0.415	0.573	118
DET	0.825	0.922	0.871	205
F	0.952	0.899	0.925	199
IMF	0.000	0.000	0.000	1
IMP	0.000	0.000	0.000	1
INDF	0.867	0.812	0.839	32
INF	0.879	0.802	0.839	172
M	0.922	0.924	0.923	459
M/N	0.000	0.000	0.000	4
N	0.820	0.432	0.566	95
NOM	0.931	0.860	0.894	442
NOM/ACC	0.133	0.143	0.138	14
NOM/ACC/DAT	0.000	0.000	0.000	1
NO_GLOSS	0.803	0.943	0.867	510
PART	0.919	0.840	0.877	81
PASS	0.000	0.000	0.000	1

Continued on next page

Feature / Metric	Precision	Recall	F1-score	Support / Value
PAST	0.000	0.000	0.000	5
PL	0.750	0.158	0.261	19
POSS	0.000	0.000	0.000	8
PRO	0.991	0.846	0.913	534
REL	0.000	0.000	0.000	1
SG	0.955	0.919	0.937	1207
Overall Metrics				
Accuracy (subset)				0.810
Precision (macro)				0.614
Recall (macro)				0.516
F1-score (macro)				0.540
Precision (micro)				0.905
Recall (micro)				0.871
F1-score (micro)				0.888
Hamming loss				0.019

Note. During evaluation, unknown classes were ignored by the scorer: [ART].

8.5 User stories

As a...	I would like to...	So that...
Researcher	1. Automate TGT whenever a session folder is uploaded for a production study 2. Have a list of available languages and their regions 3. Insert [no audio] in TGT if there's no audible response 4. Show more info in the app's message window during the task 5. Apply the app to non-Labvanced data	1. Monitor task quality from the start 2. Ensure the testing language is suitable before data collection 3. Avoid manually checking empty audio 4. Adjust immediately in case of interruptions 5. Process manually cut recordings as usual
Research Assistant	1. Make the process easier, faster, and user-friendly 2. Annotate sessions already on Drive to check status of transcription, translation, or glossing	1. Avoid opening sessions just to check status
Research Assistant	1. Use the app in the browser 2. Improve speed 3. See logs 4. Log progress in real time	1. Easier, safer usage 2. Faster processing 3. Instant error checking and file verification 4. Monitor progress via percentage or binary count
Research Assistant	1. Use the app in the browser 2. View log data	1. Use app on any device without installing software 2. Understand performance and duration
Research Assistant	1. Auto-convert Labvanced output to project format 2. Easy learning curve 3. Connect to OneDrive for cloud file access	1. Save time and effort
Lab Manager	1. Automate glossing while respecting study-specific needs (e.g., minimal vs. maximal glossing)	1. Save time on annotation 2. Ensure consistency to prevent misunderstandings or logical errors
Lab Manager	1. Automate transcription, translation, and glossing 2. Produce outputs that closely match our manual annotation guidelines	1. Save RA time and ensure consistency across studies

TABLE 8.5: Users stories gathered from stakeholders.

8.6 Requirements fulfillment

ID	Requirement Description / Fulfillment Strategy	Status
F1	Advanced raw data is accepted as standard input	Implemented
F2	No other input formats currently supported; straightforward to extend	Not implemented
F3	Linguistic tasks implemented modularly	Implemented
F4	Training subsystem integrated into backend	Implemented
F5	No current mechanism for detecting or flagging missing data	Not implemented
F6	Logging implemented throughout all major processing stages	Implemented
F7	Required columns are created automatically during processing	Implemented
F8	No metadata or folder-tagging mechanism implemented on OneDrive	Not implemented
NF1	System runs fully locally on secure server	Implemented
NF2	Web-based interface and Docker container provided	Implemented
NF3	Hosted on local server with acceptable performance on large sessions	Implemented
NF4	Tested on Windows and macOS via Docker / browser	Implemented
NF5	Always available on local server	Implemented
NF6	User interface designed with Builder.io to prioritize usability	Implemented
NF7	OneDrive integration using Microsoft Graph API and MSAL	Implemented

TABLE 8.6: Mapping of system requirements to implementation details and fulfillment status.

Bibliography

- Ahia, Orevaoghene et al. (2023). "Do All Languages Cost the Same? Tokenization in the Era of Commercial Language Models". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, pp. 9904–9923. DOI: [10.18653/v1/2023.emnlp-main.614](https://doi.org/10.18653/v1/2023.emnlp-main.614).
- Alenezi, Mamdouh and Mohammed Akour (2025). "AI-Driven Innovations in Software Engineering: A Review of Current Practices and Future Directions". In: *Applied Sciences* 15.3, p. 1344. ISSN: 2076-3417. DOI: [10.3390/app15031344](https://doi.org/10.3390/app15031344).
- Bain, Max et al. (2023). *WhisperX: Time-Accurate Speech Transcription of Long-Form Audio*. DOI: [10.21437/Interspeech.2023-78](https://doi.org/10.21437/Interspeech.2023-78).
- Bajraktari, Adrian, Michelle Binder, and Andreas Vogelsang (2024). "Requirements Engineering for Research Software: A Vision". In: *2024 IEEE 32nd International Requirements Engineering Conference (RE)*. IEEE, 423–431. DOI: [10.1109/re59067.2024.00050](https://doi.org/10.1109/re59067.2024.00050).
- Bangor, Aaron, Philip Kortum, and James Miller (2009). "Determining what Individual SUS Scores Mean: Adding an Adjective Rating Scale". In: *Journal of Usability Studies* 4.3, pp. 114–123.
- Beck, Kent et al. (2001). *Manifesto for Agile Software Development*. Accessed: 30.11.2025. URL: <https://agilemanifesto.org/>.
- Black, H. Andrew and Gary F. Simons (2006). "The SIL FieldWorks Language Explorer Approach to Morphological Parsing". In: *Texas Linguistics Society 10: Computational Linguistics for Less-Studied Languages*. Ed. by Nicholas Gaylord et al. Accessed: 30.11.2025. Stanford, CA: CSLI Publications, pp. 37–66. URL: https://web.stanford.edu/group/cslipublications/cslipublications/TLS/TLS10-2006/TLS10_Black_Simons.pdf.

- Blasi, Damián E. et al. (2022). “Over-reliance on English Hinders Cognitive Science”. In: *Trends in Cognitive Sciences* 26.12, pp. 1153–1170. ISSN: 1364-6613, 1879-307X. DOI: [10.1016/j.tics.2022.09.015](https://doi.org/10.1016/j.tics.2022.09.015).
- Boersma, Paul and David Weenink (2001). “Praat, a System for Doing Phonetics by Computer”. In: *Glott International* 5.9/10, pp. 341–345.
- Bostrom, Kaj and Greg Durrett (2020). “Byte Pair Encoding is Suboptimal for Language Model Pretraining”. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. Ed. by Trevor Cohn, Yulan He, and Yang Liu. Online: Association for Computational Linguistics, pp. 4617–4624. DOI: [10.18653/v1/2020.findings-emnlp.414](https://doi.org/10.18653/v1/2020.findings-emnlp.414).
- Brooke, John (1996). “SUS: A ‘Quick and Dirty’ Usability Scale”. In: *Usability Evaluation in Industry*. Ed. by Patrick W. Jordan et al. London: Taylor & Francis, pp. 189–194.
- Comrie, Bernard, Martin Haspelmath, and Balthasar Bickel (2008). *The Leipzig Glossing Rules: Conventions for Interlinear Morpheme-by-Morpheme Glosses*. Max Planck Institute for Evolutionary Anthropology & University of Leipzig. URL: <https://www.eva.mpg.de/lingua/pdf/Glossing-Rules.pdf>.
- Costa-jussà, Marta R. et al. (2022). *No Language Left Behind: Scaling Human-Centered Machine Translation*. arXiv: [2207.04672](https://arxiv.org/abs/2207.04672) [cs.CL]. URL: <https://arxiv.org/abs/2207.04672>.
- Elsner, Micha and David Liu (2025). “Prompt and Circumstance: A Word-by-Word LLM Prompting Approach to Interlinear Glossing for Low-Resource Languages”. In: *Proceedings of the 22nd SIGMORPHON Workshop on Computational Morphology, Phonology, and Phonetics*. Albuquerque, New Mexico, USA: Association for Computational Linguistics, pp. 1–14. DOI: [10.18653/v1/2025.sigmorphon-main.1](https://doi.org/10.18653/v1/2025.sigmorphon-main.1).
- Fan, Angela et al. (Jan. 2021). “Beyond english-centric multilingual machine translation”. In: *J. Mach. Learn. Res.* 22.1. ISSN: 1532-4435. DOI: [10.48550/arXiv.2010.11125](https://doi.org/10.48550/arXiv.2010.11125).
- Felderer, Michael et al. (2025). “Investigating Research Software Engineering: Toward RSE Research”. In: *Communications of the ACM* 68.2, pp. 20–23. DOI: [10.1145/3685265](https://doi.org/10.1145/3685265).

- Finger, Holger et al. (2017). "LabVanced: A Unified JavaScript Framework for Online Studies". In: *Proceedings of the 2017 International Conference on Computational Social Science (IC2S2)*. Cologne, Germany. URL: https://www.labvanced.com/static/2017_IC2S2_LabVanced.pdf.
- Foley, Ben et al. (2018). "Building Speech Recognition Systems for Language Documentation: The CoEDL Endangered Language Pipeline and Inference System (ELPIS)". In: *6th Workshop on Spoken Language Technologies for Under-Resourced Languages (SLTU 2018)*. ISCA, pp. 205–209. DOI: [10.21437/SLTU.2018-43](https://doi.org/10.21437/SLTU.2018-43).
- Ginn, Michael, Mans Hulden, and Alexis Palmer (2024). "Can we teach language models to gloss endangered languages?" In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. Ed. by Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen. Miami, Florida, USA: Association for Computational Linguistics, pp. 5861–5876. DOI: [10.18653/v1/2024.findings-emnlp.337](https://doi.org/10.18653/v1/2024.findings-emnlp.337).
- Henrich, Joseph, Steven J. Heine, and Ara Norenzayan (2010). "The weirdest people in the world?" In: *Behavioral and Brain Sciences* 33.2-3, pp. 61–83. ISSN: 1469-1825, 0140-525X. DOI: [10.1017/S0140525X0999152X](https://doi.org/10.1017/S0140525X0999152X).
- Hofmann, Valentin, Janet B. Pierrehumbert, and Hinrich Schütze (2021). "The Word is Not Enough: On the Compositionality of Language Models". In: *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. Association for Computational Linguistics, pp. 3634–3648. DOI: [10.18653/v1/2021.eacl-main.318](https://doi.org/10.18653/v1/2021.eacl-main.318).
- Honnibal, Matthew et al. (2017). *spaCy: Industrial-Strength Natural Language Processing in Python*. Accessed: 30.11.2025. URL: <https://spacy.io>.
- Joshi, Pratik et al. (2020). "The State and Fate of Linguistic Diversity and Inclusion in the NLP World". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Ed. by Dan Jurafsky et al. Online: Association for Computational Linguistics, pp. 6282–6293. DOI: [10.18653/v1/2020.acl-main.560](https://doi.org/10.18653/v1/2020.acl-main.560).
- Katz, Daniel S. and Simon Hettrick (2023). "Research Software Engineering in 2030". In: *2023 IEEE 19th International Conference on e-Science (e-Science)*.

- Limassol, Cyprus: IEEE, pp. 1–2. DOI: [10.1109/e-Science58273.2023.10254813](https://doi.org/10.1109/e-Science58273.2023.10254813).
- Le, Thanh-Thien, Linh The Nguyen, and Dat Quoc Nguyen (2024). “PhoWhisper: Automatic Speech Recognition for Vietnamese”. In: *Proceedings of the Second Tiny Papers Track at ICLR 2024*. URL: <https://arxiv.org/abs/2406.02555>.
- LeibnizDream Project (2021). *Realizing Leibniz’s Dream: Child Languages as a Mirror of the Mind (LeibnizDream)*. Accessed: 30.11.2025. URL: <https://leibnizdream.eu/>.
- Marneffe, Marie-Catherine de et al. (2021). “Universal Dependencies”. In: *Computational Linguistics* 47.2, pp. 255–308. ISSN: 0891-2017. DOI: [10.1162/coli_a_00402](https://doi.org/10.1162/coli_a_00402).
- Pohl, Klaus and Chris Rupp (2015). *Requirements Engineering Fundamentals: A Study Guide for the Certified Professional for Requirements Engineering Exam – Foundation Level, IREB-compliant*. en. 2nd ed. Rocky Nook Computing. Santa Barbara, CA: Rocky Nook. ISBN: 978-1-937538-77-4.
- Qi, Peng et al. (2020). “Stanza: A Python Natural Language Processing Toolkit for Many Human Languages”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. Ed. by Asli Celikyilmaz and Tsung-Hsien Wen. Online: Association for Computational Linguistics, pp. 101–108. DOI: [10.18653/v1/2020.acl-demos.14](https://doi.org/10.18653/v1/2020.acl-demos.14).
- Radford, Alec et al. (2023). “Robust Speech Recognition via Large-Scale Weak Supervision”. In: *Proceedings of the 40th International Conference on Machine Learning*. Ed. by Andreas Krause et al. Vol. 202. Proceedings of Machine Learning Research. Accessed: 30.11.2025. PMLR, pp. 28492–28518. URL: <https://proceedings.mlr.press/v202/radford23a.html>.
- Royce, Winston W. (1987). “Managing the development of large software systems: concepts and techniques”. In: *Proceedings of the 9th International Conference on Software Engineering*. ICSE ’87. IEEE Computer Society Press, 328–338. ISBN: 0897912160.

- Scheliga, Kaja et al. (2019). *Dealing with research software: Recommendations for best practices*. Technical Report. Helmholtz Open Science Coordination Office / Open Science Working Group of the Helmholtz Association. DOI: [10.2312/os.helmholtz.003](https://doi.org/10.2312/os.helmholtz.003).
- Schrepp, Martin, Jessica Kollmorgen, and Jörg Thomaschewski (2023). “A Comparison of SUS, UMUX-LITE, and UEQ-S”. In: *Journal of User Experience* 18.2, pp. 86–104. DOI: [10.5555/3604890.3604893](https://doi.org/10.5555/3604890.3604893).
- SIL, International (2022). *Linguistics Software*. Accessed: 30.11.2025. URL: <https://www.sil.org/linguistics/linguistics-software>.
- SIL International (2024a). *Field Linguist’s Toolbox: Language Data Management Software*. Accessed: 30.11.2025. URL: <https://software.sil.org/toolbox/>.
- (2024b). *FieldWorks Language Explorer™: Dictionary Creation Software*. Accessed: 30.11.2025. URL: <https://software.sil.org/fieldworks/>.
- (2025). *SayMore - Language Development Software*. Accessed: 30.11.2025. URL: <https://software.sil.org/saymore/>.
- Singh, Siddharth et al. (2023). “An Open-source Web-based Application for Development of Resources and Technologies in Underresourced Languages”. In: *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*. Ed. by Liling Tan et al. Singapore: Association for Computational Linguistics, pp. 120–129. DOI: [10.18653/v1/2023.nlposs-1.14](https://doi.org/10.18653/v1/2023.nlposs-1.14).
- The Language Archive (2024). *ELAN: EUDICO Linguistic Annotator*. Version 7.0. Accessed: 30.11.2025. URL: <https://archive.mpi.nl/tla/elan>.
- Tiedemann, Jörg and Santhosh Thottingal (2020). “OPUS-MT – Building open translation services for the World”. In: *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation*. Ed. by André Martins et al. Accessed: 30.11.2025. Lisboa, Portugal: European Association for Machine Translation, pp. 479–480. URL: <https://aclanthology.org/2020.eamt-1.61/>.
- Weissweiler, Leonie et al. (2023). “Counting the Bugs in ChatGPT’s Wugs: A Multilingual Investigation into the Morphological Capabilities of a Large Language Model”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Ed. by Houda Bouamor, Juan Pino, and

- Kalika Bali. Singapore: Association for Computational Linguistics, pp. 6508–6524. DOI: [10.18653/v1/2023.emnlp-main.401](https://doi.org/10.18653/v1/2023.emnlp-main.401).
- Wittenburg, Peter et al. (2006). “ELAN: a Professional Framework for Multimodality Research”. In: *Proceedings of the Fifth International Conference on Language Resources and Evaluation (LREC’06)*. Ed. by Nicoletta Calzolari et al. Accessed: 30.11.2025. Genoa, Italy: European Language Resources Association (ELRA). URL: <https://aclanthology.org/L06-1082/>.